

Lastenheft

Projekt: BugTracker

Projektbezeichnung	BugTracker – Webanwendung zur Fehlerverwaltung
Kurs	Software Engineering II
Hochschule	MCI Management Center Innsbruck
Lehrender / Auftraggeber	Prof. Andrea Corradini
Gruppe	Gruppe DE 3
Projektleitung	
Erstellt am	April 2026
Team	Amort Julian-Alexander, Demianov Oleksandr, Gulenko Maksym, Lamesic Patrick, Widner Josef
Erstellt am	26.04.2026
Letzte Änderung am	26.04.2026
Status	in Bearbeitung
Aktuelle Version	1.0

Änderungshistorie

Nr.	Datum	Version	Art der Änderung	Verantwortlich
1	24.04.2026	0.1	Erstellung	Lamesic Patrick
2	26.04.2026	0.5	Überarbeitung: Personas, FA-Tabelle erweitert (FA-12 zu Muss, FA-14 neu), NFA messbar gemacht, Use Cases UC-04 bis UC-06 ergänzt	Gulenko Maksym
3	27.04.2026	1.0	Finalisierung: Lieferumfang strukturiert, Abnahmekriterien in 8.1–8.4 gegliedert, Tech-Stack auf Spring Boot + React präzisiert	Gulenko Maksym
4	27.04.2026	1.1	FA-Tabelle erweitert (FA-15 neu, FA-16 neu)	Widner Josef

1. Einleitung

Das vorliegende Lastenheft beschreibt die Anforderungen des Auftraggebers an die zu entwickelnde Webanwendung „BugTracker“. Ziel des Projekts ist die Umsetzung einer einfachen Anwendung, mit der Softwarefehler (Bugs) strukturiert erfasst, verwaltet und bis zur Behebung nachverfolgt werden können. Das Dokument dient als Grundlage für das anschließend zu erstellende Pflichtenheft sowie für die spätere Umsetzung.

Auftraggeber des Projekts ist Prof. Andrea Corradini im Rahmen des Kurses Software Engineering II am MCI Innsbruck. Das Dokument bildet die Grundlage für das nachfolgende Pflichtenheft (Übungszettel ÜZ3, Frist 29.04.2026) sowie für die Implementierungs- und Bereitstellungsphase (ÜZ4 bis ÜZ6, Mai 2026).

2. Allgemeines

2.1 Ziel und Zweck des Dokuments

Dieses Lastenheft beschreibt die fachlichen und technischen Anforderungen an den BugTracker aus Sicht des Auftraggebers. Es bildet die Grundlage für das Pflichtenheft und dient dem Entwicklungsteam als Orientierung während der gesamten Projektlaufzeit.

2.2 Ausgangssituation

Die zu adressierende Zielsituation ist ein kleines bis mittelgroßes Software-Entwicklungsteam (5–20 Personen), das aktuell ohne dediziertes Bug-Tracking-Werkzeug arbeitet. In der Praxis werden Fehlermeldungen heute über folgende Kanäle erfasst:

- E-Mail an einzelne Entwicklerinnen oder Entwickler
- Chat-Nachrichten in Slack, MS Teams oder Discord
- Mündliche Meldung im Daily Stand-up
- Notizen in einzelnen Word- oder Excel-Dateien

Daraus resultierende Probleme:

- Meldungen gehen verloren oder werden vergessen
- Der Bearbeitungsstand ist intransparent („Wer kümmert sich gerade?“)
- Doppelmeldungen werden nicht erkannt
- Es existiert keine Historie über erledigte Fehler — Wissen geht verloren
- Priorisierung erfolgt willkürlich

Bestehende Profi-Lösungen wie Jira, GitHub Issues, Bugzilla oder Linear wären technisch geeignet, scheiden aber wegen Lizenzkosten, Komplexität oder Vendor-Lock-in für das Zielszenario aus. Es wird daher eine schlanke, selbstgehostete Eigenlösung angestrebt.

2.3 Projektbezug

Das Projekt wird im Rahmen eines Universitätskurses als eigenständiges Vorhaben umgesetzt. Die Entwicklung erfolgt in einem kleinen Team. Der Umfang ist bewusst begrenzt, da der Fokus des Kurses nicht auf der Programmierung selbst liegt, sondern auf dem Durchlauf eines strukturierten Softwareentwicklungsprozesses.

2.4 Abkürzungen

- Bug: Softwarefehler bzw. Fehlverhalten einer Anwendung
- UI: User Interface, Benutzeroberfläche
- FA: Funktionale Anforderung
- NFA: Nicht-funktionale Anforderung
- MoSCoW: Methode zur Priorisierung von Anforderungen (Must / Should / Could / Won't)
- UC: Use Case, Anwendungsfall
- MVP: Minimum Viable Product
- CRUD: Create, Read, Update, Delete
- JPA: Jakarta Persistence API (im Projekt: nicht verwendet, stattdessen Spring JDBC)
- REST: Representational State Transfer
- SPA: Single Page Application

3. Konzept

3.1 Produktvision

Der BugTracker ist eine schlanke Webanwendung, die es Nutzenden ermöglicht, Fehler in einem Softwareprodukt zu melden, einzusehen, zu bearbeiten und als erledigt zu markieren. Die Anwendung soll intuitiv bedienbar sein und ohne lange Einarbeitung genutzt werden können.

3.2 Nutzen der Anwender

- Zentrale Sammelstelle für Fehlermeldungen statt verstreuter E-Mails und Chatnachrichten
- Klare Nachvollziehbarkeit des Bearbeitungsstands jedes Bugs
- Eindeutige Zuordnung von Verantwortlichkeiten
- Schneller Überblick über offene, in Bearbeitung befindliche und erledigte Fehler

3.3 Zielgruppen / Personas

Persona 1 — Marie, die Entwicklerin

- 28 Jahre, Backend-Entwicklerin in einem 8-köpfigen Team, drei Jahre Berufserfahrung
- Bekommt heute Fehlermeldungen per E-Mail, Slack und im Daily — vergisst regelmäßig welche
- Will: klare Liste, was sie als Nächstes zu tun hat; einen Ort statt fünf
- Pain Points: doppelte Bug-Reports, keine Priorisierung, Kontextverlust beim Tool-Wechsel

Persona 2 — Tom, der Tester

- 24 Jahre, Werkstudent in der QA, ein Jahr Erfahrung
- Findet täglich neue Fehler, muss sie strukturiert dokumentieren
- Will: schnelles Erfassen mit klaren Reproduktionsschritten; sehen ob ein Bug schon gemeldet wurde
- Pain Points: weiß nie ob ein Bug schon existiert; Entwickler fragen ständig nach Details

Persona 3 — Sandra, die Projektleiterin

- 35 Jahre, Tech Lead, verantwortlich für Stakeholder-Kommunikation
- Muss dem Kunden über Bug-Status berichten können, Sprintplanung machen
- Will: schnellen Überblick „Wie viele offene Bugs? Welche kritisch?“
- Pain Points: muss aktuell jeden Entwickler einzeln fragen, was Status ist

3.4 Abgrenzung

Nicht Bestandteil des Projekts sind:

- Integration mit externen Systemen wie Jira, GitHub Issues oder E-Mail-Servern
- Komplexes Rechte- und Rollenmanagement mit vielen Nutzerrollen
- Mobile Apps (iOS/Android)
- Automatische Fehlererkennung oder Logging-Anbindung
- Mehrsprachigkeit der Oberfläche über Deutsch hinaus
- Komplexes Rechte- und Rollenmanagement mit feingranularen Permissions
- Kanban- oder Sprint-Boards mit Drag-and-Drop
- Zeiterfassung pro Bug
- Multi-Tenancy (eine Installation = ein Team)
- Versionierung der Bug-Beschreibung (Edit-Historie pro Feld)
- Externe REST-API für Drittsysteme

4. Funktionale Anforderungen

Die folgende Tabelle listet die funktionalen Anforderungen auf. Jede Anforderung besitzt eine eindeutige ID der Form FA-xx. Die Priorisierung erfolgt nach der MoSCoW-Methode und ist in der separaten MoSCoW-Matrix detailliert dargestellt.

ID	Titel	Beschreibung	Prio.
FA-01	Bug anlegen	Nutzende können einen neuen Bug mit Titel, Beschreibung, Priorität und Status erfassen.	Muss
FA-02	Bugliste anzeigen	Eine Übersicht aller erfassten Bugs wird in einer Liste angezeigt.	Muss
FA-03	Bug im Detail ansehen	Ein einzelner Bug kann mit allen Feldern in einer Detailansicht geöffnet werden.	Muss
FA-04	Bug bearbeiten	Ein vorhandener Bug kann nachträglich bearbeitet werden (z. B. Status oder Beschreibung).	Muss
FA-05	Bug archivieren	Ein Bug kann aus der aktiven Liste in ein Archiv verschoben werden. Hartes Löschen ist nicht vorgesehen, um den Audit-Trail zu erhalten.	Muss
FA-06	Status verwalten	Jedem Bug kann ein Status zugewiesen werden. Erlaubte Werte: NEU, IN_BEARBEITUNG, IM_REVIEW, ERLEDIGT, ABGELEHNT, ARCHIVIERT. Übergänge folgen einer State Machine (Details im Pflichtenheft).	Muss
FA-07	Priorität setzen	Jedem Bug kann eine Priorität zugewiesen werden (Niedrig, Mittel, Hoch, Kritisch).	Soll
FA-08	Bearbeiter zuweisen	Einem Bug kann eine verantwortliche Person als	Soll

ID	Titel	Beschreibung	Prio.
		Bearbeiter zugewiesen werden.	
FA-09	Nach Status filtern	Die Bugliste kann nach dem Status gefiltert werden.	Soll
FA-10	Nach Titel suchen	In der Bugliste kann nach dem Titel eines Bugs gesucht werden.	Kann
FA-11	Kommentarfunktion	Zu jedem Bug können Kommentare hinzugefügt werden.	Kann
FA-12	Benutzeranmeldung	Nutzende können sich mit Benutzernamen und Passwort anmelden.	Muss
FA-13	E-Mail-Benachrichtigung	Bei Änderungen an einem Bug wird eine E-Mail an den Bearbeiter gesendet.	Wird nicht
FA-14	Bug-Historie	Zu jedem Bug ist die Änderungshistorie sichtbar (wer hat wann was geändert).	Soll
FA-15	Rollen-basierte Zugriffskontrolle	Drei Rollen werden unterschieden: TESTER, DEVELOPER, ADMIN. Bestimmte Aktionen (z. B. Projekte/Tags verwalten) sind ausschließlich ADMIN vorbehalten.	Muss
FA-16	Tags verwalten	ADMIN-Nutzer können Tags mit Name und Farbe anlegen, bearbeiten und löschen. Tags werden Bugs zugewiesen.	Soll

5. Nicht-funktionale Anforderungen

5.1 Allgemeine Anforderungen

- Die Bedienung soll ohne schriftliche Anleitung für Personen mit grundlegender PC-Erfahrung möglich sein.
- Die Darstellung erfolgt auf Desktop-Bildschirmen; eine mobile Optimierung ist nicht erforderlich.

5.2 Technische Anforderungen

- Programmiersprache: Java
- Framework: Spring Boot
- Datenspeicherung: relationale Datenbank (z. B. H2 oder PostgreSQL)
- Zugriff über einen gängigen Webbrowser (Chrome, Firefox, Edge)
- Ausführung lokal bzw. auf einem einfachen Server; kein produktiver Hosting-Betrieb vorgesehen
- Backend: Java 21, Spring Boot 3.x mit Spring Web, Spring Data JDBC, Spring Security
- Frontend: React 18+ mit TypeScript
- Datenbank: PostgreSQL (Dev und Prod)
- Build-Tools: Maven (Backend), Vite (Frontend)
- Versionsverwaltung: Git, Repository auf GitHub
- Zugriff: über gängige Webbrowser (Chrome, Firefox, Edge — Stand 2026)
- Ausführung: lokal bzw. auf einem einfachen Server

5.3 Qualitätsanforderungen

ID	Kategorie	Beschreibung	Prio.
NFA-01	Benutzbarkeit	Ein neuer Nutzer mit grundlegender PC-Erfahrung muss in der Lage sein, einen Bug innerhalb von 3 Minuten ohne externe Hilfe anzulegen. Nachweis: Usability-Test mit 3 Probanden.	Soll
NFA-02	Technologie	Backend in Java 21 mit Spring Boot 3.x; Frontend in React mit TypeScript.	Muss
NFA-03	Plattform	Die Anwendung läuft auf einem Standard-PC und wird über einen Webbrowser bedient.	Muss
NFA-04	Antwortzeit	Einfache Aktionen (Anlegen, Anzeigen, Bearbeiten) reagieren in weniger als 2 Sekunden bei bis zu 100 Bugs. Messung mit JMeter, lokal auf Standard-Laptop (16 GB RAM, Intel i5 oder besser).	Soll
NFA-05	Wartbarkeit	Code-Coverage durch Unit-Tests $\geq 60\%$ in Service-Schicht. SonarLint-Analyse ohne Critical-Findings. JavaDoc auf öffentlichen Methoden in Service- und Controller-Schicht.	Soll
NFA-06	Zuverlässigkeit	Die Anwendung darf bei normaler Nutzung nicht unkontrolliert abstürzen; Daten dürfen nicht verloren gehen.	Muss
NFA-07	Sicherheit	Passwörter werden mit BCrypt gehasht. Session-Cookies sind HttpOnly und Secure-Flag-aktiv. SQL-Injection ist durch ausschließliche Verwendung von PreparedStatements ausgeschlossen.	Muss
NFA-08	Architektur	Trennung von Backend (Spring Boot REST-API) und Frontend (React SPA); Kommunikation ausschließlich über JSON-REST. Frontend kann unabhängig deployed werden.	Muss

6. Beispielhafte Anwendungsfälle

UC-01: Neuen Bug melden

Feld	Inhalt
Akteur	Tester (Persona Tom) oder Entwickler (Persona Marie)
Vorbedingung	Akteur ist eingeloggt
Auslöser	Akteur hat einen Fehler in der Software entdeckt

Hauptszenario	1. Akteur klickt im Hauptmenü auf „Neuer Bug“. 2. System zeigt Erfassungsformular mit Pflichtfeldern Titel und Beschreibung sowie optionalen Feldern Priorität und Status. 3. Akteur füllt mindestens die Pflichtfelder aus. 4. Akteur klickt „Speichern“. 5. System validiert die Eingaben. 6. System speichert den Bug mit Status NEU und aktuellem Zeitstempel. 7. System leitet auf die Bugliste weiter; der neue Bug ist sichtbar.
Alternativfluss 4a	Pflichtfeld fehlt → System markiert das Feld, zeigt Fehlermeldung, Speichern wird nicht ausgeführt → Akteur korrigiert → weiter bei Schritt 4.
Nachbedingung	Bug ist in der Datenbank persistiert, Status = NEU, Reporter ist der eingeloggte User.
Akzeptanzkriterium	UC-01 ist in unter 5 Sekunden Gesamtzeit ausführbar.

UC-02: Bug bearbeiten

Feld	Inhalt
Akteur	Entwickler (Persona Marie)
Vorbedingung	Akteur ist eingeloggt; Bug existiert in der Datenbank
Auslöser	Akteur möchte den Status oder Inhalt eines Bugs aktualisieren
Hauptszenario	1. Akteur öffnet einen Bug aus der Liste. 2. System zeigt Detailansicht. 3. Akteur klickt „Bearbeiten“. 4. Akteur ändert Felder (z. B. Status auf IN_BEARBEITUNG, ergänzt die Beschreibung). 5. Akteur klickt Speichern. 6. System validiert und persistiert; Änderung wird in der Bug-Historie geloggt. 7. Detailansicht zeigt aktualisierte Werte.
Alternativfluss 4a	Ungültiger Statusübergang (z. B. ERLEDIGT → NEU) → System zeigt Fehlermeldung → Akteur wählt anderen Status.
Nachbedingung	Bug ist aktualisiert; Eintrag in Bug-Historie existiert.

UC-03: Bug abschließen

Feld	Inhalt
Akteur	Entwickler (Persona Marie)
Vorbedingung	Bug existiert mit Status IN_BEARBEITUNG oder IM_REVIEW
Auslöser	Fehler wurde behoben und verifiziert

Hauptszenario	1. Akteur öffnet den Bug in der Detailansicht. 2. Akteur setzt den Status auf ERLEDIGT. 3. Akteur klickt Speichern. 4. System protokolliert die Statusänderung in der Historie. 5. Detailansicht zeigt Status ERLEDIGT.
Nachbedingung	Bug bleibt in der Liste sichtbar, ist aber als abgeschlossen markiert und kann archiviert werden.

UC-04: Bugs filtern und durchsuchen

Feld	Inhalt
Akteur	Projektleiterin (Persona Sandra)
Vorbedingung	Akteur ist eingeloggt; in der Datenbank existieren Bugs
Auslöser	Akteur möchte sich einen schnellen Überblick zu einem bestimmten Status oder Thema verschaffen
Hauptszenario	1. Akteur öffnet die Bugliste. 2. Akteur wählt im Status-Filter z. B. „NEU“ und „IN_BEARBEITUNG“ aus. 3. System filtert die Liste sofort und zeigt nur Bugs mit ausgewähltem Status. 4. Akteur gibt im Suchfeld einen Begriff ein (z. B. „Login“). 5. System filtert zusätzlich nach Titel-Treffern. 6. Akteur kopiert die URL und teilt sie mit dem Team.
Nachbedingung	Liste zeigt nur die gefilterten Treffer; URL ist teilbar (Filter im Query-String).

UC-05: Login und Logout

Feld	Inhalt
Akteur	Beliebiger registrierter Nutzer
Vorbedingung	Nutzer hat einen Account
Auslöser	Nutzer möchte die Anwendung verwenden bzw. die Sitzung beenden
Hauptszenario (Login)	1. Nutzer öffnet die Anwendung. 2. System zeigt Login-Seite. 3. Nutzer gibt Benutzernamen und Passwort ein. 4. System validiert die Credentials. 5. Bei Erfolg: Weiterleitung zur Bugliste, Session-Cookie ist gesetzt.
Alternativfluss 4a	Falsche Credentials → System zeigt Fehlermeldung „Login fehlgeschlagen“, ohne preiszugeben, ob der Benutzername existiert.

Hauptszenario (Logout)	Nutzer klickt auf „Logout“ → Session wird invalidiert → Weiterleitung zur Login-Seite.
Nachbedingung	Bei Login: Session aktiv, geschützte Seiten erreichbar. Bei Logout: keine Session, geschützte Seiten redirecten zur Login-Seite.

UC-06: Bearbeiter zuweisen

Feld	Inhalt
Akteur	Projektleiterin (Persona Sandra) oder Entwicklerin (Persona Marie)
Vorbedingung	Bug existiert; mindestens ein registrierter User ist vorhanden
Auslöser	Verantwortlichkeit für einen Bug muss zugeordnet werden
Hauptszenario	1. Akteur öffnet den Bug in der Detailansicht. 2. Akteur wählt im Bearbeiter-Dropdown einen User aus. 3. Akteur klickt Speichern. 4. System persistiert die Zuweisung; Bearbeiter erscheint in Liste und Detail. 5. Eintrag in der Bug-Historie wird erzeugt.
Alternativfluss 2a	Akteur wählt „Kein Bearbeiter“ → Bug.assigneeId wird auf null gesetzt; Spalte zeigt „—“.
Nachbedingung	Bug hat einen zugewiesenen Bearbeiter (oder keinen); Änderung in Historie geloggt.

7. Lieferumfang

Liefergegenstand	Format
Lauffähige Webanwendung „BugTracker“	Spring Boot Backend (JAR) + React Frontend (statische Dateien)
Quellcode	Git-Repository auf GitHub
Lastenheft	Word- oder PDF-Dokument
Pflichtenheft	Word- oder PDF-Dokument
MoSCoW-Matrix	Excel (.xlsx)
README.md mit Setup-Anleitung	Markdown im Repo

Unit-Tests + Integrations-Tests	JUnit / Spring Boot Test
DevOps-Pipeline-Konfiguration	GitHub Actions YAML
Abschlusspräsentation	PowerPoint / PDF
Projektbericht	Word- oder PDF-Dokument

8. Abnahmekriterien

Das Produkt gilt als abgenommen, wenn folgende Kriterien erfüllt sind:

8.1 Funktional

- Alle als „Muss“ priorisierten Anforderungen sind implementiert und in Live-Demo fehlerfrei vorführbar
- Anwendungsfälle UC-01 bis UC-06 laufen in der Live-Demo ohne Fehler durch
- Alle als „Soll“ priorisierten Anforderungen sind zu mindestens 80 % implementiert

8.2 Qualität

- Code-Coverage in der Service-Schicht: $\geq 60\%$ (nachgewiesen durch JaCoCo-Report)
- SonarLint-Analyse: keine Critical-Findings
- Manuelle Usability-Tests mit 3 Probanden ohne kritische Auffälligkeiten bestanden

8.3 Dokumentation

- Quellcode vollständig im GitHub-Repository
- README.md mit Setup-Anleitung und Architekturskizze vorhanden
- Lastenheft v1.0 und Pflichtenheft v1.0 als PDF abgegeben
- BugTracker-Anwendung enthält mindestens 5 Beispiel-Bugs für die Demo

8.4 Präsentation

- Abschlusspräsentation am 22.05.2026 erfolgreich durchgeführt
- Projektbericht eingereicht bis 22.05.2026, 23:59 Uhr